Kick Start 2020 - Round B

Analysis: Bike Tour

For each of the checkpoints, we can determine if it is a peak in O(1) time by comparing its height to the heights of the checkpoints before and after it.

There are **N** checkpoints, so the total time complexity of this approach is O(**N**), which is sufficient for both Test Set 1 and Test Set 2.

Sample Code(C++)

```
int countPeaks(vector<int> checkpoints) {
  int peaks = 0;
  for(int i = 1; i < checkpoints.size() - 1; i++) {
     if(checkpoints[i-1] < checkpoints[i] && checkpoints[i+1] < checkpoints[i]) {
        peaks++;
     }
  }
  return peaks;
}
```

Kick Start 2020 - Round B

Analysis: Bus Routes

We need to take the buses in order from 1 to **N**. Let the buses be B_1, B_2, ..., $B_{\mathbf{N}}$. Also, let us define a *good starting day* to be any day Y in the range [1..**D**] such that it is possible to start the journey on day Y and take all buses in the order from 1 to **N** before the end of day **D**. Note that we do not require Y to be a multiple of $\mathbf{X_1}$, so there may be some waiting time involved in the beginning of the journey.

For a fixed day Y, let us see if it is a good starting day or not. The best strategy would be to take bus B_1 as early as possible on or after day Y. This is because it would give us more days to take subsequent buses. Let us say we took bus B_1 on day D_1. Now the best strategy would be to take bus B_2 as early as possible on or after day D_1. Thus, if we took bus B_i on day D_i, it would be optimal to take bus B_{i+1} as early as possible on or after day D_i.

Now we need to find out what is the earliest possible day for Bucket to take bus B_i on or after a particular day K. Since bus B_i only runs on days that are multiples of $\mathbf{X_i}$, we need to find the smallest multiple of $\mathbf{X_i}$ greater than or equal to K. This can be calculated using the formula $\lceil K / \mathbf{X_i} \rceil * \mathbf{X_i}$. Thus if bus B_i is taken on day D_i, then it would be optimal to take bus B_{i+1} on day $D_{i+1} = \lceil D_i / \mathbf{X_{i+1}} \rceil * \mathbf{X_{i+1}}$. Thus, day Y is a good starting day if $D_{\mathbf{N}} \leq \mathbf{D}$, and this question can be answered in O(**N**) time.

Test set 1

D can be at most 100, so we can find the latest good starting day by using the above approach for each day Y in the range [1..**D**]. The time complexity of this naive algorithm is O(**DN**).

Test set 2

Now **D** can be at most 10^{12}, so the naive algorithm would time out. Consider the largest good starting day P. Obviously, any day before P would be good as well because we can take the buses on the same days as if we started the journey on day P. Because of this observation, we can binary search on the range from 1 to **D** to find the largest good starting day P. The time complexity of the solution is O(**N** log **D**). Note that we can reduce the time complexity to O(**N** log(**D**/$\mathbf{X_1}$)) by restricting the search to multiples of $\mathbf{X_1}$ only.

Alternate solution

It is possible to solve the problem in linear time by working out the solution backwards. If we want to start our journey as late as possible, we should try to take the last bus $B_{\mathbf{N}}$ as late a possible, namely, on day $D_{\mathbf{N}}$, which is the largest multiple of $\mathbf{X_N}$, less than or equal to day **D**. Similarly, in order to be on time for the last bus on day $D_{\mathbf{N}}$, we have to take bus $B_{\mathbf{N}-1}$ no later than on day $D_{\mathbf{N}-1}$, which is the largest multiple of $\mathbf{X_{N-1}}$, less than or equal to $D_{\mathbf{N}}$. In general, bus B_i should be taken no later than on day D_i, which is the largest multiple of $\mathbf{X_i}$, less than or equal to D_{i+1}. The last calculated value D_1 is the answer to the problem.

Note that the largest multiple of $\mathbf{X_i}$ that occurs before a day L can be calculated in constant time as L - L mod $\mathbf{X_i}$. Therefore, the overall time complexity of this solution is O(**N**).

Kick Start 2020 - Round B

Analysis: Robot Path Decoding

To facilitate parsing of the program, let us define *ClosingBracket*(i) as the index of the closing bracket corresponding to the opening bracket at index i. We can find *ClosingBracket*(i) for each opening bracket using a stack in linear time, which is similar to checking whether a string is a correct bracket sequence or not, see this article for more details.

Test Set 1

The total number of moves is limited by 10^4 per test. Consider the expanded version of a program P to be the string consisting of characters N,S,W,E only and having the same moves as P. For example, the program 2(3(N)EW) would expand to NNNEWNNNEW. Since the number of moves is small, we can generate the expanded program, and calculate the position of the robot easily by taking one step at a time.

For an original subprogram between indices L and R, the equivalent expanded version *Expanded*(L, R) can be constructed recursively as follows. We start with an empty string *Result*, iterate the subprogram from the left index L to the right index R, and consider two cases:

- If the i-th symbol is in {'N','S','E','W'}, append it to *Result*.
- If the i-th symbol is a digit D then
  - Call *Expanded*(i+2, *ClosingBracket*(i+1)-1) to construct the expanded version P' of the next subprogram recursively,
  - Append P' to *Result* D times, and
  - Advance the current position i to *ClosingBracket*(i+1)+1.

The first case takes constant time. In the second case, it takes O(D × |P'|) time to append the subprogram P' to the result D times. Let LEN be the length of the expanded program. The total expanded length of the subprograms at the second nesting level is at most LEN/2. The total expanded length of subprograms at the third nesting level is at most LEN/4, and so on. Thus the time complexity would be bounded by LEN + LEN / 2 + LEN / 4 + LEN / 8 + .. which is equal to 2 × LEN as this is a geometric progression. Hence, the time complexity to generate the expanded version of the original program would be O(LEN).

Test Set 2

Now it is possible that the number of moves is exponential in the length of the original program. Thus it would be impossible to execute the moves one by one in the given time.

For the ease of explanation, let us assume that the rows and columns are numbered from 0 (inclusive) to 10^9 (exclusive). Suppose that the robot is at position (a, b) and now we come across instruction X(Y) in the program. Let us say subprogram Y changes the current position of the robot by dx, dy (because of the torus shape of Mars, we can assume that $0 \le dx < 10^9$ and $0 \le dy < 10^9$). Then the position of the robot after following the instruction X(Y) would be $((a + X * dx) \bmod 10^9, (b + X * dy) \bmod 10^9)$ as the subprogram Y is repeated X times. Hence, we just need to find the relative displacement of the robot by each subprogram.

For a subprogram between indices L and R, the relative displacement of the robot can be calculated using *Evaluate*(L, R) recursively as follows. Consider that we are currently at the square (a, b), which is initially the square (0, 0). Iterate the subprogram from the left index L to the right index R, and consider two cases:

- If the i-th symbol is in {'N','S','E','W'}, change the current position of the robot accordingly.
- If the i-th symbol is a digit D then
    - Call *Evaluate*(i+2, *ClosingBracket*(i+1)-1)to get the relative displacement (dx, dy) of the robot by the next subprogram recursively,
    - Change the current position to ((a + D * dx) mod 10^9, (b + D * dy) mod 10^9), and
    - Advance the current position i to *ClosingBracket*(i+1)+1.

Clearly, we visit each character in the program exactly once. Hence, the time complexity of the solution is O(N), where N is the length of the program.

Kick Start 2020 - Round B

Analysis: Wandering Robot

Test Set 1

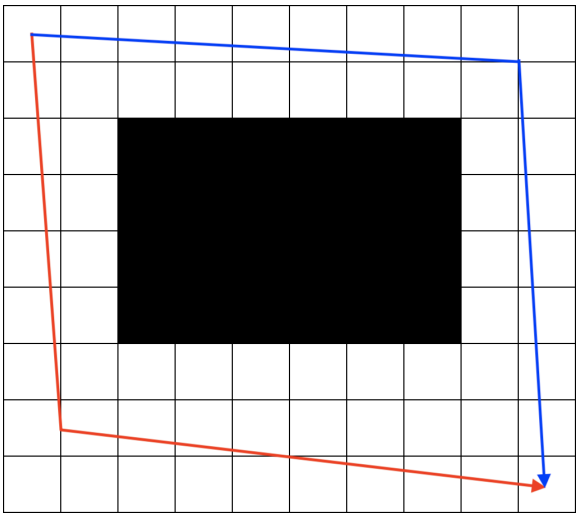
We can solve this test set using dynamic programming. Let f(x, y) be the probability Jemma passes the challenge if she is currently in the square (x, y). The base case of this function is f(**W**, **H**) = 1. Also, if the square (x, y) has been removed, then f(x, y) = 0.

If there is only one possible square to go to from square (x, y) (i.e. either x = **W** or y = **H**), then f(x, y) = f(x', y'), where (x', y') is the possible next square. Otherwise, let (x_1', y_1') and (x_2', y_2') be the possible next squares. Since they have the same probability to become the next square, f(x, y) = 0.5 × f(x_1', y_1') + 0.5 × f(x_2', y_2').

The running time and space of this solution is O(**W** × **H**).

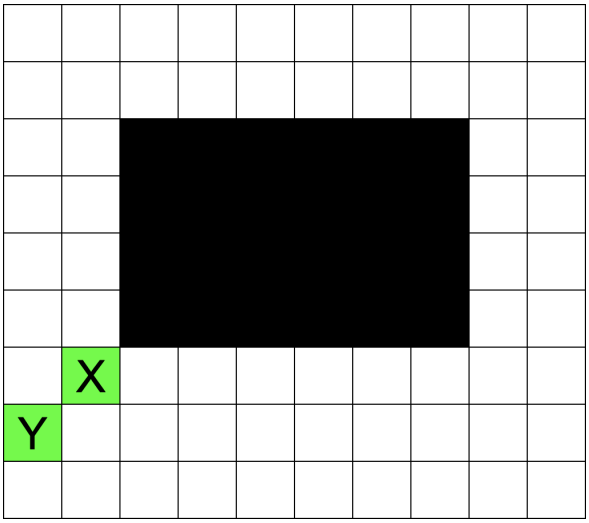
Test Set 2

The first observation to solve this problem is to realize that there are two ways to avoid the hole: either going to the left and the bottom of the hole (illustrated by the red path in the figure below), or going to the top and the right of the hole (illustrated by the blue path in the figure below).



It can be seen that the set of paths in the red path and the blue path are disjoint--there is no path that goes both to the left of the hole and to the top of the hole simultaneously. Therefore, we can compute the probability that Jemma passes the challenge by taking the red path and the blue path separately and compute the sum of both probabilities.

Since the probability of passing the challenge by taking the blue path can be computed similarly, we only focus on computing the probability of passing the challenge by taking the red path for the rest of the discussion. The next observation to solve this problem is that we can choose a set of squares diagonally from the bottom-left corner of the hole (illustrated by the green squares below) such that Jemma has to pass exactly one of the squares to pass the challenge by taking the red path. Also, by landing on one of the squares, it is no longer possible that Jemma will fall to the hole, thus passing the challenge by taking the red path is now guaranteed.



Therefore, computing the probability of passing the challenge by taking the red path is equivalent to computing the probability that Jemma will land on one of the green squares. Similar to the red and blue paths discussion, since Jemma cannot pass two green squares simultaneously, we can compute the probability that Jemma lands on each square separately and compute the sum of all probabilities.

Let us take square X for an example. Consider all paths that go to the square X. For each move in the path, there is a 0.5 probability that the move will follow the path. Since the number of moves to square X is (**L** + **D** - 2), there is a $(0.5)^{(\mathbf{L} + \mathbf{D} - 2)}$ probability that this path will be taken. This number has to be multiplied with the number of paths to go to square X, which can be computed using a single binomial coefficient. The probability of reaching any particular green square is the same for all but the green square in the last row, which is left to the reader as an exercise.

To handle floating point issues, we can store every huge number in their log representation (i.e. storing $\log_2(x)$ instead of x). We can then compute the value of $C(n, k) / 2^n$ using $2^{\log_2(n! / (k! \times (n - k)!) / 2^n)} = 2^{\log_2(n!) - \log_2(k!) - \log_2((n - k)!) - n}$, which takes constant time to compute if we have precomputed every value of $\log_2(x!)$. Since there can be at most O(N) green squares, where N is the larger length of the grid (i.e. N = max(**H**, **W**)), the total running time of this solution is O(N).